

# file-ownership

## File Ownership — Why Downloads Landed Unowned, and How It Is Fixed

**Revision:** 1 **Last modified:** 2026-08-21T14:38:55Z **Audience:** Operators running boba on a rootless-podman host who have seen downloaded files they cannot open, move, or delete without changing ownership first. **Authority:** Feature 002-user-owned-downloads. Measured evidence in [specs/002-user-owned-downloads/research.md](#) R1, R4, and the R9 correction record.

---

### The defect

Every file the stack downloaded landed owned by an identity you do not have.

```
$ ls -l /mnt/DATA/some-download/  
-rw-r--r-- 1 100999 100999 1234567 Aug 21 12:00 file.mkv
```

100999 has no entry in /etc/passwd, so tools render it as UNKNOWN:UNKNOWN. The practical consequence was not cosmetic. You could not move, rename, delete, or in many cases even *read* your own downloads without first taking ownership of them by hand — one `chown` per download, forever. Directories were worse: the operator's own `stat` on a file inside them returned Permission denied.

The same defect reached beyond downloads. `config/` held 51 items owned by 100999, including `config/boba.db` at mode 600. Because `docs/BOBA_DATABASE.md` instructs you to back up `boba.db` and `.env` together, and warns that losing the master key is unrecoverable, that documented backup procedure **could not be performed at all**.

## Why it happened

Nothing was misconfigured in the ordinary sense. This is a consequence of how **rootless** containers map identities, meeting a setting that is correct advice everywhere else.

Under rootless podman your account is granted a block of subordinate UIDs:

```
$ grep "^$(whoami):" /etc/subuid
milosvasic:100000:65536
```

Read that as: *“this user may use 65536 UIDs starting at 100000.”* The kernel then maps identities between the container and the host like this:

| Inside the container | On the host                | Why                        |
|----------------------|----------------------------|----------------------------|
| uid <b>0</b> (root)  | <b>your own uid</b> (1000) | The namespace owner is you |
| uid <b>1</b>         | 100000                     | first subordinate uid      |
| uid <b>N</b>         | <b>100000 + N - 1</b>      | the general rule           |

**Worked example.** The linuxserver.io images take a PUID environment variable and run the application as that uid. The stack set PUID=1000 — which looks obviously right, since the operator *is* uid 1000 on the host. But 1000 is interpreted **inside the container**, so the mapping applies:

$$100000 + 1000 - 1 = 100999$$

Every file the application wrote therefore landed on the host as uid 100999. The setting that would be correct under rootful Docker is precisely the one that breaks things under rootless podman.

The counter-intuitive corollary is the whole fix:

Under rootless podman, **container root is you**. A container running as uid 0 writes files owned by your host account — not by host root.

## The fix, per service

Two services needed changing. Three were already correct and were deliberately left alone.

| Service     | Image          | Change         | Why |
|-------------|----------------|----------------|-----|
| qbittorrent | linuxserver.io | PUID=0, PGID=0 |     |

| Service              | Image              | Change         | Why                                                                  |
|----------------------|--------------------|----------------|----------------------------------------------------------------------|
|                      |                    |                | Makes the app run as container root → host uid 1000                  |
| jackett              | linuxserver.io     | PUID=0, PGID=0 | Same                                                                 |
| download-proxy       | python:3.12-alpine | <b>none</b>    | Already runs as container root; measured writing host-uid-1000 files |
| qbittorrent-proxy-go | built Go           | <b>none</b>    | No USER directive in its Dockerfile, so it runs as root              |
| boba-jackett         | built Go           | <b>none</b>    | Already runs as container root; measured writing host-uid-1000 files |

## PUID=0 does not give the container host privilege

This is the objection worth answering directly, because “set it to root” reads like a security regression and is not one here. The container’s uid 0 **is** your unprivileged host account, uid 1000. It holds no capability on the host that your own shell does not already hold. The stack remains fully rootless; nothing runs under sudo, and no system-wide service is introduced. What changed is which *identity inside the namespace* the application adopts — and that identity maps to you.

## Why userns\_mode: keep-id is not used

--userns=keep-id is the other well-known way to line up container and host identities, and it is the wrong tool for this stack:

- On the **linuxserver.io images it hangs the container** with no output. Those images boot as root to run their s6 init before dropping to PUID; under keep-id there is no usable root, and the image stalls instead of failing fast. This was measured twice.
- On the **three services that already run as root it is pointless** — they were already writing host-uid-1000 files. Adding keep-id would have changed nothing beneficial while introducing the same no-usable-root failure mode.

**usersns\_mode: keep-id must not be added to any service in this stack.**

## Verify it yourself

Do not verify this by reading docker-compose.yml. A configuration file states an intention; it does not prove what the running container actually writes. Write a real file through the real service and read the owner back from the host:

```
cd /path/to/boba
DL="{QBITTORRENT_DATA_DIR:-/mnt/DATA}"

podman exec qbittorrent sh -c 's6-setuidgid abc touch /
    downloads/.ownership-probe'
stat -c '%u (%U)' "$DL/.ownership-probe"
rm -f "$DL/.ownership-probe"
```

Expected output — your own uid and username:

```
1000 (milosvasic)
```

If you see 100999 (UNKNOWN), the fix is not in effect on the running container. Note that a PUID change lives in docker-compose.yml, and **a restart does not re-read the compose file** — you need `./start.sh --recreate`.

You can check the other services the same way; they mount `./config`:

```
podman exec qbittorrent-proxy sh -c 'touch /config/.ownership-
    probe'
podman exec boba-jackett sh -c 'touch /config/.ownership-
    probe2'
stat -c '%n -> %u (%U)' config/.ownership-probe config/.ownership-
    probe2
rm -f config/.ownership-probe config/.ownership-probe2
```

## Prove your probe can actually see a failure

A clean reading only means something if the same probe would have reported a dirty one. To confirm the probe is not simply blind, run it against a throwaway container configured the old, broken way:

```
T=$(mktemp -d "$PWD/tmp/ownprobe.XXXX")
podman run --rm -e PUID=1000 -e PGID=1000 -v "$T":/downloads \
    lscr.io/linuxserver/qbittorrent:latest \
    sh -c 's6-setuidgid abc touch /downloads/control'
stat -c '%u (%U)' "$T/control"          # as you: Permission
    denied
```

```
podman unshare stat -c 'in-ns uid %u' "$T/control" # in-ns uid
1000 = host 100999
podman unshare rm -rf "$T"
```

The Permission denied is the defect seen from your side. The in-namespace reading of 1000 is the same file seen from the container's side, and  $100000 + 1000 - 1$  resolves it to host 100999. A probe that produces this result on a broken configuration, and 1000 (yourname) on the running stack, is a probe you can trust.

Run this in a scratch directory, never against your real download tree — the linuxserver init takes ownership of whatever it is given as /downloads.

## Content that is already wrongly owned

Fixing the configuration stops *new* files from landing unowned. It does nothing to the files already on disk — those keep their 100999 ownership until something changes it.

A repair tool is provided at **scripts/ownership\_repair.sh**. It walks the in-scope locations (the download tree and the container-written project paths such as config/), records what it is about to change before changing it, reports progress so a long run stays distinguishable from a hang, and resumes where it left off if interrupted. It blocks download-writing services while it runs, so a repair and an active download cannot race.

**Its command-line interface is documented in [specs/002-user-owned-downloads/contracts/repair-cli.md](#)** — consult that for flags, exit codes, and invocation. This guide deliberately does not restate them.

If you prefer to repair a single path by hand, the underlying operation is a chown executed inside the user namespace, which is what makes the mapping resolve correctly:

```
podman unshare chown -R 0:0 /path/to/thing
```

Inside the namespace that is uid 0; on the host it is your own uid 1000. Verify with `stat -c '%u (%U)'` afterwards rather than assuming it worked.

## What not to change

- Do not set PUID/PGID back to 1000 on qbittorrent or jackett. That reintroduces the defect in full: every future download becomes unowned again.
- Do not add `usersns_mode: keep-id` to any service.

- Do not add USER directives to the Go images — they run as container root by design, which under rootless podman is the operator.
- After any change to docker-compose.yml, use ./start.sh --recreate. A plain restart does not re-read the file, and has previously produced a false “fixed”.

## See also

- [specs/002-user-owned-downloads/research.md](#) — R1 (root cause), R4 (why PUID=0 works), R9 (the correction that removed keep-id from the plan).
- [specs/002-user-owned-downloads/contracts/repair-cli.md](#) — the repair tool’s interface.
- [docs/BOBA\\_DATABASE.md](#) — the boba.db + .env backup procedure this defect blocked.